

1. Introduction to Artificial Intelligence

Meaning of Artificial Intelligence

Artificial Intelligence is a branch of computer science that deals with creating machines or computer programs that can perform tasks which normally require human intelligence.

In simple words:

AI means making machines intelligent.

AI systems can:

- Think
- Learn
- Reason
- Solve problems
- Make decisions
- Understand language
- Recognize images
- Act according to the situation

1.2. Simple Definition of AI

Artificial Intelligence is the study of how to make computers perform tasks that require intelligence when performed by humans.

Examples:

AI System	What it does
ChatGPT	Answers questions and generates text
Google Maps	Finds best route
Alexa / Siri	Understands voice commands
Self-driving car	Drives automatically
Medical AI	Helps in disease diagnosis
Face recognition	Identifies a person from image

1.3. Why Do We Need AI?

AI is needed because many real-world problems are complex and require fast, intelligent decision-making.

AI helps in:

- Reducing human effort
- Solving complex problems
- Making faster decisions
- Improving accuracy
- Automating repetitive tasks
- Handling large amounts of data
- Supporting humans in risky areas

Example:

A doctor may take time to analyze many medical reports, but an AI system can quickly support disease prediction by analyzing past patient data.

1.4. Main Goals of AI

The main goals of Artificial Intelligence are:

Goal	Explanation
Reasoning	Ability to think logically and make decisions
Learning	Ability to improve from experience or data
Problem solving	Ability to find solutions to difficult problems
Perception	Ability to understand images, sounds, and environment
Natural language understanding	Ability to understand human language
Planning	Ability to decide steps to achieve a goal
Robotics	Ability to control machines or robots intelligently

1.5. Human-Like Reasoning vs Rational Reasoning

AI can be understood in two important ways.

Human-Like Reasoning

Human-like reasoning means making machines think or behave like humans.

Example:

A chatbot talking like a human.

Here, the focus is on copying human behavior.

Rational / Optimal Reasoning

Rational reasoning means making machines choose the best possible action to achieve a goal.

Example:

Google Maps selecting the shortest or fastest route.

Here, the focus is not just behaving like humans, but making the best decision.

Human-Like Reasoning	Rational / Optimal Reasoning
Tries to imitate humans	Tries to choose the best action
Focuses on human behavior	Focuses on correct decision
May behave like humans	May perform better than humans
Example: chatbot	Example: Google Maps, A* search

Important point:

In AI, rational reasoning is very important because an AI system should select the best action based on the situation.

1.6. Applications of AI

AI is used in many fields today.

Area	AI Application
Games	Chess, video games, strategy games

Area	AI Application
Robotics	Industrial robots, medical robots, agricultural robots
Search engines	Google search, Bing search
Healthcare	Disease prediction, medical image analysis
Chatbots	Customer support, education, personal assistants
Transportation	Self-driving cars, traffic prediction
Education	AI tutors, automatic grading
Agriculture	Crop disease detection, pest prediction
Banking	Fraud detection, credit scoring
Security	Face recognition, surveillance systems

1.7. Difference Between AI, ML, and DL

Students should clearly understand that **AI, ML, and DL are related, but not the same.**

Artificial Intelligence

AI is the broad field of making machines intelligent.

Example:

Expert systems, robots, search algorithms, chatbots.

Machine Learning

Machine Learning is a part of AI.

It allows machines to learn from data.

Example:

Spam email detection.

Deep Learning

Deep Learning is a part of Machine Learning.

It uses artificial neural networks with many layers.

Example:

Face recognition, speech recognition, image classification.

Relationship

Artificial Intelligence



Machine Learning



Deep Learning

2.Intelligent Agent in AI

In the realm of AI, Intelligent Agents drive automation and decision-making with cognitive abilities.

- Intelligent agents represent a subset of AI systems demonstrating intelligent behaviour, including adaptive learning, planning, and problem-solving.
- It operate in dynamic environments, where it makes decisions based on the information available to them.
- These agents dynamically adjust their behaviour, learning from past experiences to improve their approach and aiming for accurate solutions.

The design of an intelligent agent typically involves four key components:

1. **Perception:** Agents have sensors or mechanisms to observe and perceive aspects of their environment. This may involve collecting data from the physical world, accessing databases, or receiving input from other software components.
2. **Reasoning:** Agents possess computational or cognitive capabilities to process the information they perceive. They use algorithms, logic, or machine learning techniques to analyze data, make inferences, and derive insights from the available information.
3. **Decision-Making:** Based on their perception and reasoning, agents make decisions about the actions they should take to achieve their goals. These decisions are guided by predefined objectives, which may include optimizing certain criteria or satisfying specific constraints.
4. **Action:** Agents execute actions in their environment to affect change and progress towards their goals. These actions can range from simple operations, such as sending a message or adjusting parameters, to more complex tasks, such as navigating a virtual world or controlling physical devices.

Examples of Intelligent Agents include self-driving cars, recommendation systems, virtual assistants, and game-playing AI.

Rational Agents and Rationality in Decision-Making

Intelligent agents are characterized by their rationality in decision-making, which aims to attain optimal outcomes or, in uncertain scenarios, the best-expected outcome.

A rational agent can be said to those, who do the right thing, It is an autonomous entity designed to perceive its environment, process information, and act in a way that maximizes the achievement of its predefined goals or objectives. Rational agents always aim to produce an optimal solution.

Rationality in AI refers to the principle that such agents should consistently choose actions that are expected to lead to the best possible outcomes, given their current knowledge and the uncertainties present in the environment. This principle of rationality guides the behavior of intelligent agents in the following ways:

- **Perception and Information Processing:** Rational agents strive to perceive and process information efficiently to gain the most accurate understanding of their environment.
- **Reasoning and Inference:** They employ logical reasoning and probabilistic inference to make informed decisions based on available evidence and prior knowledge.
- **Decision-Making Under Uncertainty:** When faced with uncertainty, rational agents weigh the probabilities of different outcomes and choose actions that maximize their expected utility or achieve the best possible outcome given the available information.
- **Adaptation and Learning:** Rational agents adapt their behavior over time based on feedback and experience, continuously refining their decision-making strategies to improve performance and achieve their goals more effectively.

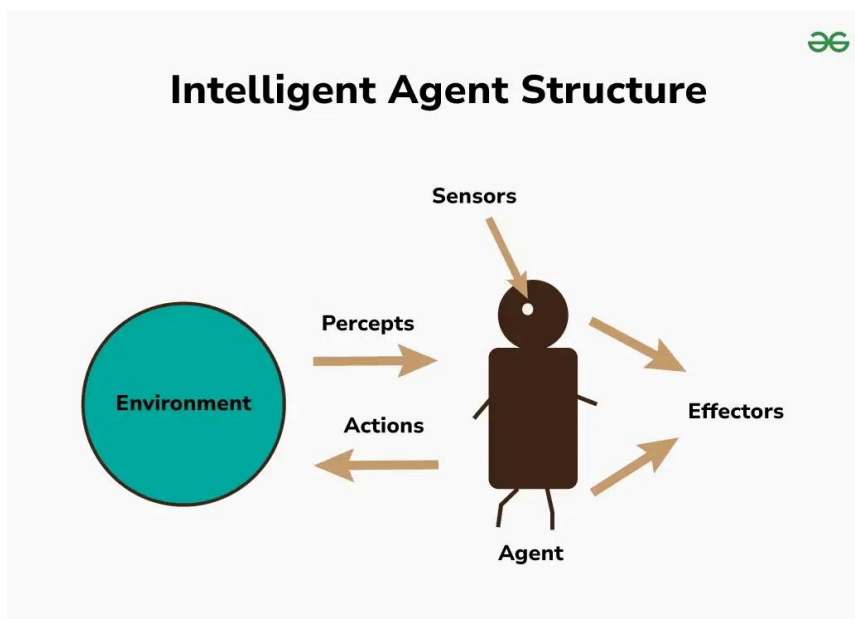
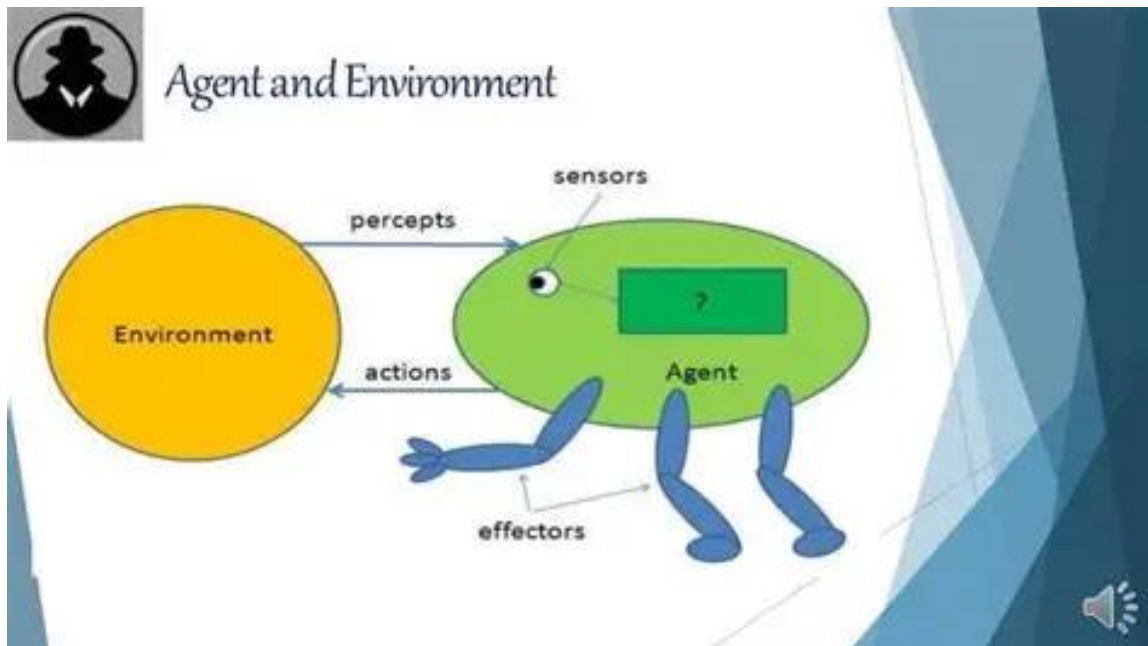
Example of a rational agent is a chess-playing AI, which selects moves with the highest likelihood of winning.

How Intelligent Agent work Inside?

An agent's internal workings involve Agent program that run on computing device and process the data comes from the environment through its

architecture. Let's discuss how an agent works from the inside using program and architecture:

1. Agent architecture



1. **Environment:** Environment is the area around the agent that it interacts with. An environment can be anything like a physical space, a room or a virtual space like a game world or the internet.
2. **Sensors:** Sensors are tools that AI agent uses to perceive their environment. They can be any physical like cameras, microphones, temperature sensors or a software sensor that read data from files.

3. **Actuators:** Actuators are tools that AI agent uses to interact with their environment through some actions. They can be any physical actuators like wheels, motors, robotic hands, or computer screens or they can be software actuators that send messages.
4. **Effectors:** Effectors take instructions from decision making mechanism and translates them into actions and these actions are performed through actuators.

2.2. Program or Decision-making mechanism:

This is the brain of the AI agent, this mechanism processes the information that is received through sensors and makes decisions through that data using programs. Let's understand how the agent's program execute the operations.

- The decision-making mechanism, often referred to as the agent's program, processes information from sensors and makes decisions based on that data.
- The program takes current percepts as input and generates actions for the actuators.
- It embodies the agent function, which maps percepts to actions based on the agent's goals and objectives.
- Various **types of agent programs** exist, such as simple reflex agents, model-based reflex agents, goal-based agents, and utility-based agents.
- These programs differ in how they process percepts and generate actions, depending on the agent's design and task requirements.

For example, a simple reflex agent may have a program that directly maps percept states to actions without considering past or future percepts for a two-state vacuum environment. This decision will be executed through effectors.

PEAS Representation of AI agent

PEAS stands for performance measure, environment, actuators and sensors. It is a framework that is used to describe an AI agent. It's a structured approach to design and understand AI systems.

1. **Performance measure:** Performance measure is a criteria that measures the success of the agent. It is used to evaluate how well the agent is achieving its goal.

For example, in a spam filter system, the performance measure could be minimizing the number of spam emails reaching the inbox.

2. **Environment:** The environment represents the domain or context in which the agent operates and interacts. This can range from physical spaces like rooms to virtual environments such as game worlds or online platforms like the internet.
3. **Actuators:** Actuators are the mechanisms through which the AI agent performs actions or interacts with its environment to achieve its goals. These can include physical actuators like motors and robotic hands, as well as digital actuators like computer screens and text-to-speech converters.
4. **Sensors:** Sensors enable the AI agent to gather information from its environment, providing data that informs its decision-making process and actions. These sensors can capture various environmental parameters such as temperature, sound, movement, or visual input. Examples of sensors include cameras, microphones, temperature sensors, and motion sensors.

Problem Solving Agents

In **artificial intelligence**, problems can be solved in a somewhat complex way and it can be necessary to use several data structures and methods. As machines help to solve problems, one of the key aspects is for example, the **Problem-Solving Agent**. What does a problem-solving agent represent and how does it work in AI systems, as well as several exciting use cases that illustrate this technology's capabilities.

What is Problem Solving Agent?

In **artificial intelligence**, issue solving means **identifying** how the **problem** is to be solved. Issues are diverse and there may be distinguished a great number of different kinds of these problems and each of them will be resolved in a particular way. The most common type of challenge that is offered to the winner of the game is a navigation puzzle or a maze.

Finding solutions to systems of equations, anticipating results, and recognizing patterns are some other categories of challenges. When it comes to problem solving, there are many methods and resources which may be used depending on the type of problem encountered.

In artificial intelligence, there are three primary processes for problem-solving:

Understanding the problem:

This is the stage of defining the details of the problem and identifying the steps that should be taken to deal with it.

Generating possible solutions:

In this stage, you will produce as many ideas as you can with reference to the details of the problem and the working of computers.

Choosing a solution:

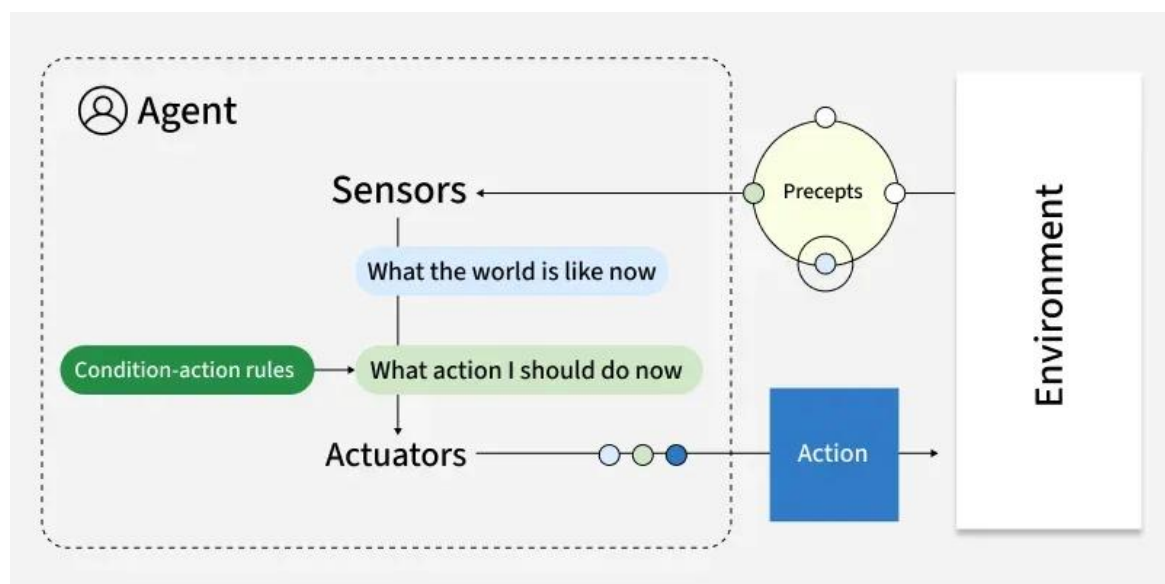
At this stage, one must identify which of the seen solutions is the best to apply because of the knowledge acquired about the problem and the methods to solve it.

2.3 Types of Agents in AI

AI agents are the foundation of many intelligent systems which helps them to understand their environment, make decisions, and act to achieve specific goals, ranging from simple rule-based to adaptive learning models.

- Operate autonomously with minimal human intervention.
- Used in applications like assistants, automation, and robotics.

1. Simple Reflex Agents

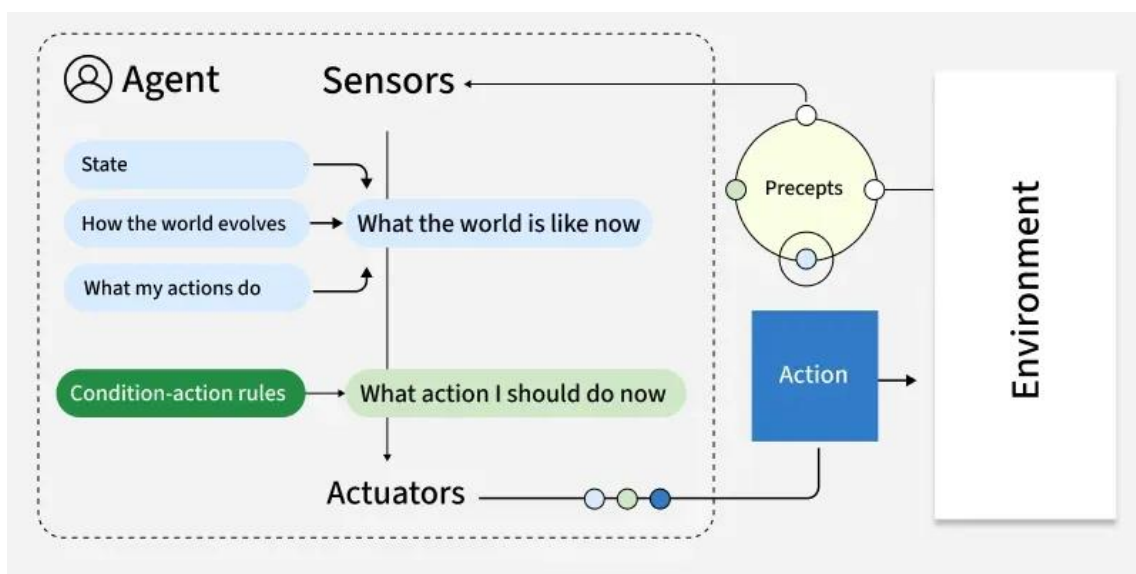


Simple reflex agents act only on the current input using predefined rules, without memory or history. They are fast and suitable for simple, fully observable environments but perform poorly in dynamic or partially observable situations due to lack of memory and reasoning.

- **Reactive:** These agents respond immediately to inputs without consideration for prior events or predicting future outcomes.
- **Limited Scope:** They excel in predictable environments where tasks are straightforward and the relationships between actions and results are well understood.
- **Quick Response:** Since decisions are made based only on immediate input, it can react without delay.
- **No Learning:** These agents cannot improve or change their behavior based on past experiences.

Example: Traffic light control systems that change signals based on fixed timing.

2. Model-Based Reflex Agents



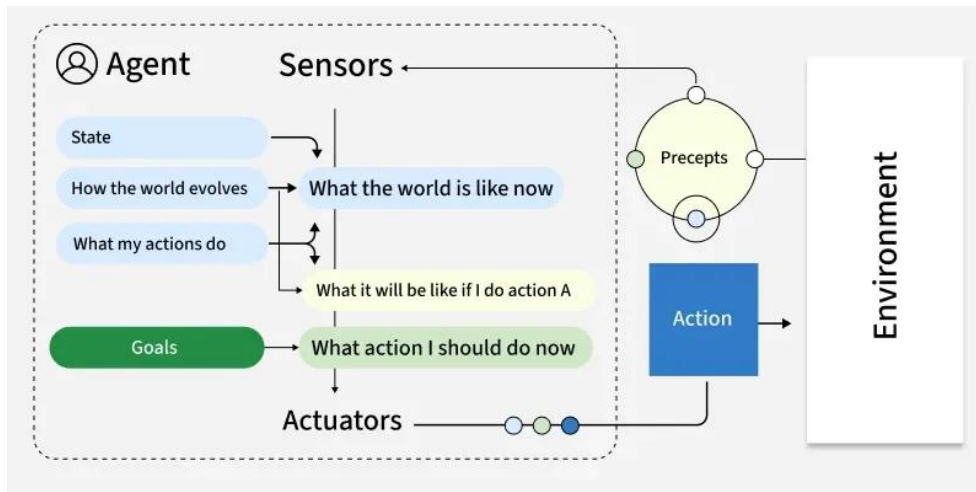
Model-based reflex agents improve simple reflex agents by maintaining an internal state of the environment to handle partially observable and changing situations. However, their decisions are still mainly reactive and depend on the accuracy of the internal model.

- **Internal State:** Maintains an internal model to handle partially observable environments and support flexible decisions.
- **Adaptive:** Updates its internal state based on new information to adjust to changes.
- **Better Decision-Making:** Uses stored information to make more informed and reliable choices.

- **Increased Complexity:** Requires more memory and computation to maintain and update the internal model.

Example: Robot vacuum cleaners that map rooms and tracks cleaned areas.

3. Goal-Based Agents

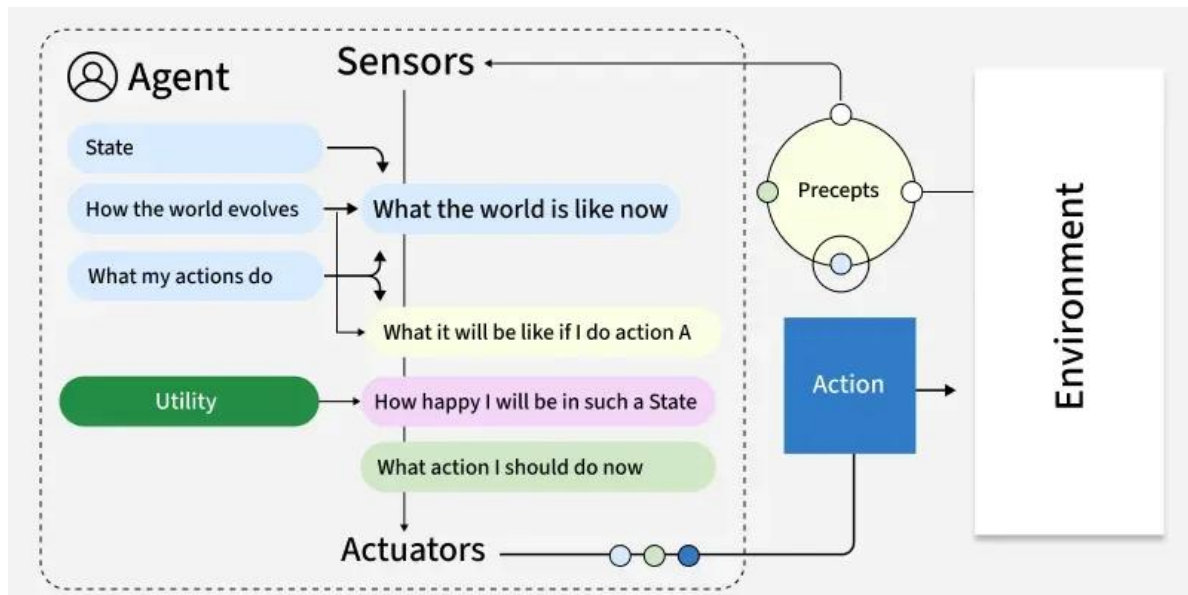


Goal-based agents choose actions by evaluating future states in relation to defined goals. They can plan action sequences to achieve objectives, enabling more flexible problem-solving, but depend on clear goals and effective planning methods in complex environments.

- **Goal-Oriented:** Makes decisions based on how well actions align with defined objectives.
- **Planning and Search:** Uses planning algorithms to evaluate possible action sequences to reach goals.
- **Flexible:** Re-plans and adjusts strategies when conditions or information change.
- **Future-Oriented:** Unlike reflex agents, they think ahead and predict future outcomes to find the best course of action.

Example: Logistics routing agents that find optimal delivery routes based on factors like distance and time. They continuously adjust to reach the most efficient route.

4. Utility-Based Agents



Utility-based agents go beyond goal-based agents by evaluating how desirable different outcomes are using a utility function. This allows them to make trade-offs and choose the most optimal action in uncertain or limited-resource situations. Their performance depends heavily on how well the utility function is designed.

- **Multi-Criteria Decision Making:** Considers multiple factors such as cost, benefit, risk, and time to choose the best action.
- **Trade-Offs:** Make decisions by balancing competing goals and preferences often finding the best "compromise."
- **Subjectivity:** Adapts to user or organizational preferences through customizable utility functions.
- **Increased Complexity:** Finding utility functions for different factors can be computationally intensive and complex.

Example: Financial portfolio management agents that evaluate investments based on factors like risk, return and diversification operate by choosing options that provide the most value.

Real-world Applications of Problem-Solving Agents in AI

Problem Solving Agents are not just theoretical concepts. These days, they are actively employed in a wide range of practical applications. Let us examine a few instances:

Gaming Agents

Problem Solving Agents are frequently utilized in video game applications. They are able to assess the condition of the game at any given time, plan out potential strategies, and execute the best play. Because of this, they may defeat human players in challenging games like Go or chess.

Robotics

Robots in warehouses and factories rely a lot on agents that can solve problems. By means of sensors, these agents sense the surroundings surrounding the robot. The robot is then programmed to follow efficient routes, and its motions and activities are coordinated accordingly.

Virtual Assistants

Goal-based agents are used to solve problems in smart home devices such as Google Home and Alexa. When you ask them for help, they can comprehend your requests, search their knowledge base for pertinent material, and give you helpful answers.

Recommendation Systems

Product recommendations from problem-solving agents are used by online businesses to make product suggestions that you might enjoy. When recommending products to you, these agents take into account your tastes and previous purchases.

3.State Space Representation

3.1 Meaning of State Space

State space representation is a way of representing a problem using different possible states and actions.

In AI, many problems are solved by moving from a **starting state** to a **goal state** using suitable actions.

Simple definition:

State space is the collection of all possible states that can occur while solving a problem.

3.2 Important Terms

Term	Meaning
Initial State	Starting point of the problem

Term	Meaning
Goal State	Final state that we want to reach
Actions / Operators	Possible moves that can be applied
Transition Model	Shows the result of applying an action
Path Cost	Cost of moving from one state to another
Solution Path	Sequence of actions from initial state to goal state

3.3 Initial State

The **initial state** is the starting condition of the problem.

Example:

In a route-finding problem, the initial state may be:

Start city = Arad

3.4 Goal State

The **goal state** is the required final condition.

Example:

In the Romania map problem:

Goal city = Bucharest

3.5 Actions / Operators

Actions are the possible moves that an agent can perform.

Example:

From Arad, possible actions may be:

- Go to Zerind
- Go to Sibiu
- Go to Timisoara

3.6 Transition Model

The **transition model** tells what happens after applying an action.

Example:

If the agent is in **Arad** and takes the action **Go to Sibiu**, then the new state becomes **Sibiu**.

State: Arad

Action: Go to Sibiu

New State: Sibiu

3.7 Path Cost

Path cost is the total cost required to reach a state.

Cost may represent:

- Distance
- Time
- Fuel
- Money
- Number of moves

Example:

If Arad to Sibiu distance is 140 km, then path cost = 140.

3.8 Solution Path

A **solution path** is the sequence of actions that takes the agent from the initial state to the goal state.

Example:

Arad → Sibiu → Fagaras → Bucharest

This is one possible solution path in the Romania map problem.

4.Search Algorithms

Search algorithms in AI are the algorithms that are created to aid the searchers in getting the right solution. The search issue contains search space, first start and end point. Now, by performing simulations of scenarios and alternatives, searching algorithms help AI agents find the optimal state for the task.

Logic used in algorithms processes the initial state and tries to get the expected state as the solution. Because of this, AI machines and applications just functioning using search engines and solutions that come from these algorithms can only be as effective as the algorithms.

AI agents can make the AI interfaces usable without any software literacy. The agents that carry out such activities do so with the aim of reaching an end goal and developing action plans that, in the end, will bring the mission to an end. Completion of the action is gained after the steps of these different actions. The AI agents find the best way through the process by evaluating all the alternatives that are present. Search systems are a common task in artificial intelligence by which you are going to find the optimum solution for the AI agents.

Problem-solving Agents:

In Artificial Intelligence, Search techniques are universal problem-solving methods. Rational agents or Problem-solving agents in AI mostly use these search strategies or algorithms to solve a specific problem and provide the best result. Problem-solving agents are the goal-based agents and use atomic representation. In this topic, we will learn various problem-solving search algorithms.

Properties of Search Algorithms:

Following are the four essential properties of search algorithms to compare the efficiency of these algorithms:

- **Completeness:** A search algorithm is said to be complete if it guarantees to return a solution if at least any solution exists for any random input.
- **Optimality:** If a solution found for an algorithm is guaranteed to be the best solution (lowest path cost) among all other solutions, then such a solution is said to be an optimal solution.
- **Time Complexity:** Time complexity is a measure of time for an algorithm to complete its task.

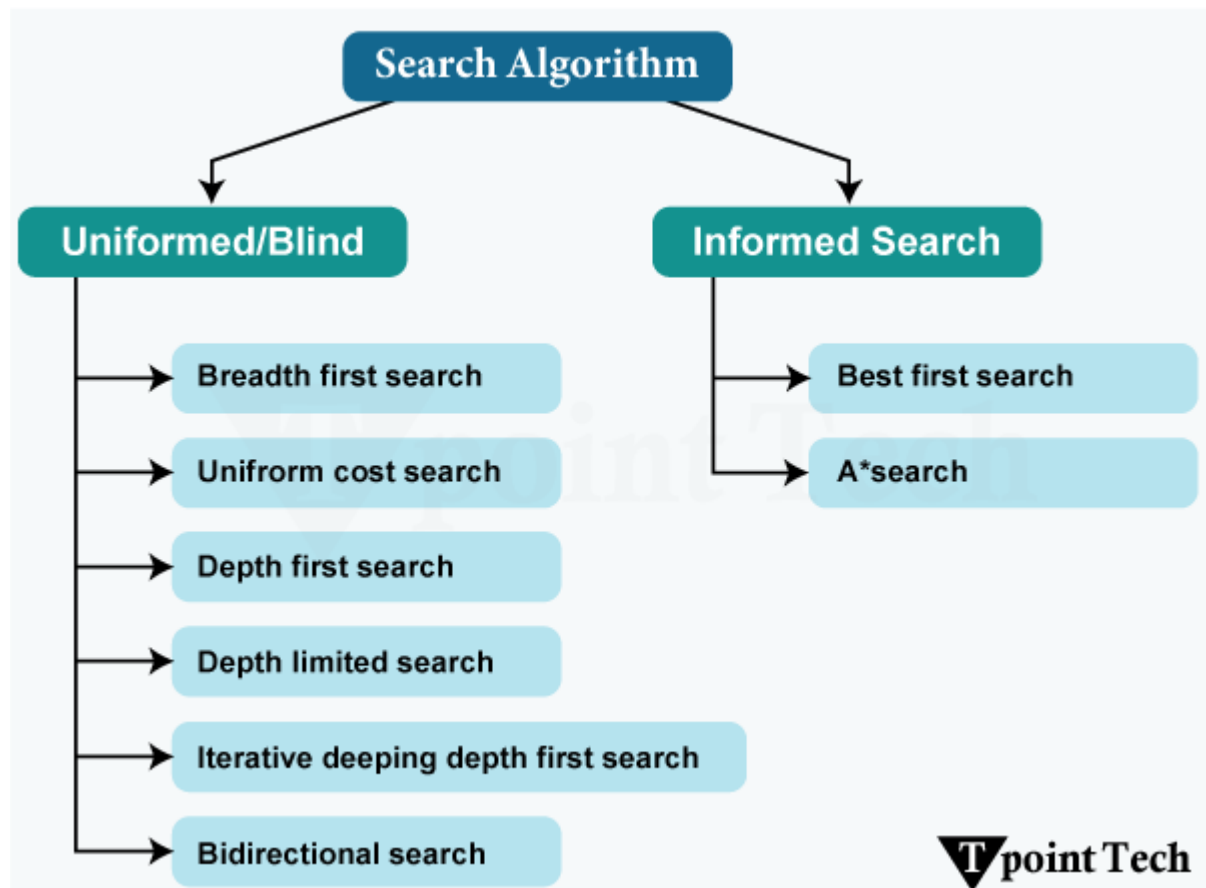
- **Space Complexity:** It is the maximum storage space required at any point during the search, as the complexity of the problem.

Search Algorithm Terminologies

- **Search:** Searching is a step-by-step procedure to solve a search problem in a given search space. A search problem can have three main factors:
 - **Search Space:** Search space represents a set of possible solutions that a system may have.
 - **Start State:** It is a state from where the agent begins the search.
 - **Goal test:** It is a function that observes the current state and returns whether the goal state is achieved or not.
- **Search Tree:** A tree representation of a search problem is called a Search tree. The root of the search tree is the root node, which corresponds to the initial state.
- **Actions:** It describes all the available actions to the agent.
- **Transition Model:** A description of what each action does can be represented as a transition model.
- **Path Cost:** It is a function that assigns a numeric cost to each path.
- **Solution:** It is an action sequence that leads from the start node to the goal node.
- **Optimal Solution:** If a solution has the lowest cost among all solutions.

Types of Search Algorithms

Based on the search problems we can classify the search algorithms into uninformed (Blind search) search and informed search (Heuristic search) algorithms.



4.1 Uninformed/Blind Search

The uninformed search does not contain any domain knowledge, such as closeness or the location of the goal. It operates in a brute-force way as it only includes information about how to traverse the tree and how to identify leaf and goal nodes. Uninformed search applies a way in which a search tree is searched without any information about the search space, like initial state operators and tests for the goal, so it is also called blind search. It examines each node of the tree until it achieves the goal node.

It can be divided into six main types:

- Breadth-first search
- Uniform cost search
- Depth-first search
- Depth limited search
- Iterative deepening depth-first search
- Bidirectional Search

4.2 Informed Search

Informed search algorithms use domain knowledge. In an informed search, problem information is available, which can guide the search. Informed search strategies can find a solution more efficiently than an uninformed search strategy. An informed search is also called a Heuristic search.

A heuristic is a way that might not always be guaranteed for the best solutions but guaranteed to find a good solution in a reasonable time.

Informed search can solve many complex problems that could not be solved in another way.

An example of informed search algorithms is a travelling salesman problem.

- Greedy Search
- A* Search

4.1 Uninformed Search Algorithms

An uninformed search is one in which the search systems do not use any clues about the suitable area, but it depends on the random nature of the Search. Nevertheless, they begin the exploration of search space (all possible solutions) synchronously. The search operation begins from the initial state and provides all possible next steps arrangements until the goal is reached.

These are mostly the simplest search strategies, but they may not be suitable for complex paths that involve irrelevant or even irrelevant components. These algorithms are necessary for solving basic tasks or providing simple processing before passing on the data to more advanced search algorithms that incorporate prioritized information.

Following are the various types of uninformed search algorithms:

1. Breadth-first Search
2. Depth-first Search
3. Depth-limited Search
4. Iterative deepening depth-first Search
5. Uniform cost search
6. Bidirectional Search

1. Breadth-first Search:

- Breadth-first Search is the most common search strategy for traversing a tree or graph. This algorithm searches breadthwise in a tree or graph, so it is called breadth-first Search.
- The BFS algorithm starts searching from the root node of the tree and expands all successor nodes at the current level before moving to nodes of the next level.
- The breadth-first search algorithm is an example of a general-graph search algorithm.
- Breadth-first Search was implemented using a FIFO queue data structure.

Advantages:

- BFS will provide a solution if any solution exists.
- If there is more than one solution for a given problem, then BFS will provide the minimal solution that requires the least number of steps.
- It also helps find the shortest path in the goal state since it needs all nodes at the same hierarchical level before moving to nodes at lower levels.
- It is also very easy to comprehend. With this, we can assign a higher rank among path types.

Disadvantages:

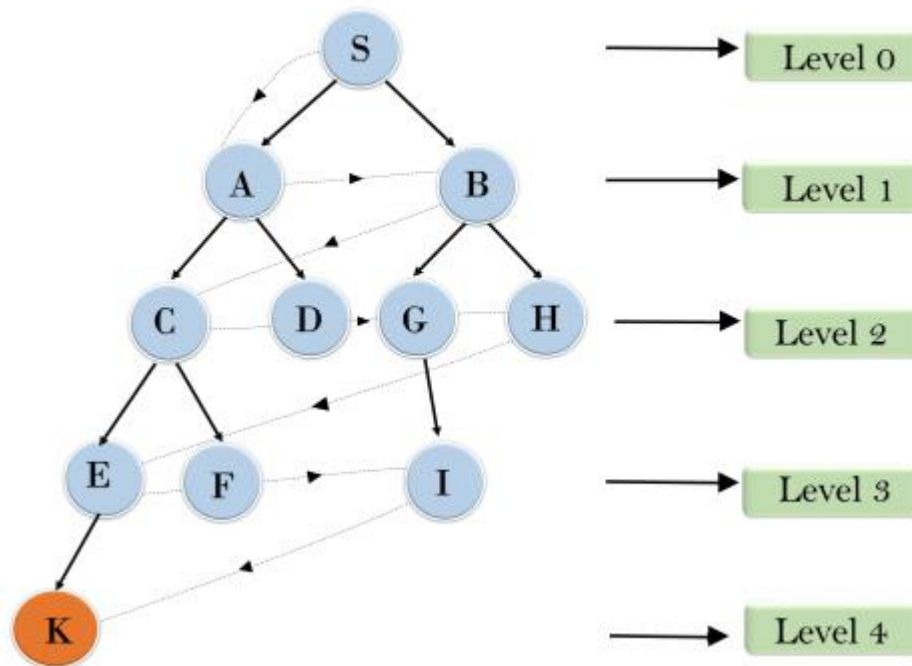
- It requires lots of memory since each level of the tree must be saved in memory to expand to the next level.
- BFS needs lots of time if the solution is far away from the root node.
- It can be a very inefficient approach for searching through deeply layered spaces, as it needs to thoroughly explore all nodes at each level before moving on to the next.

Example:

In the tree structure below, we show the traversing of the tree using the BFS algorithm from root node S to goal node K. The BFS search algorithm traverses in layers, so it will follow the path which is shown by the dotted arrow, and the traversed path will be:

S----> A---->B----->C---->D----->G---->H---->E----->F----->I----->K

Breadth First Search



Time Complexity: The time Complexity of the BFS algorithm can be obtained by the number of nodes traversed in BFS until the shallowest Node.

Where the d = depth of shallowest solution and

b is a node at every state.

$$T(b) = 1 + b^1 + b^2 + \dots + b^d = O(b^d)$$

Space Complexity: The space complexity of the BFS algorithm is determined by the memory size of the frontier, which is $O(b^d)$.

Completeness: BFS is complete, which means if the shallowest goal node is at some finite depth, then BFS will find a solution.

Optimality: BFS is optimal if path cost is a non-decreasing function of the depth of the Node.

2. Depth-first Search

- Depth-first Search is a recursive algorithm for traversing a tree or graph data structure.
- It is called the depth-first Search because it starts from the root node and follows each path to its greatest depth node before moving to the next path.

- DFS uses a stack data structure for its implementation.
- The process of the DFS algorithm is similar to that of the BFS algorithm.

Note: Backtracking is an algorithm technique that uses recursion to find all possible solutions.

Advantages:

- DFS requires very little memory as it only needs to store a stack of the nodes on the path from the root node to the current Node.
- It takes less time to reach the goal node than the BFS algorithm (if it traverses in the right path).
- With the help of this, we can store the route that is being tracked in memory to save time, as it only needs to keep one at a particular time.

Disadvantage:

- There is the possibility that many states keep re-occurring, and there is no guarantee of finding a solution.
- DFS algorithm goes for deep-down searching, and sometimes, it may go to the infinite loop.
- The depth-first Search (DFS) algorithm does not always find the shortest path to a solution.

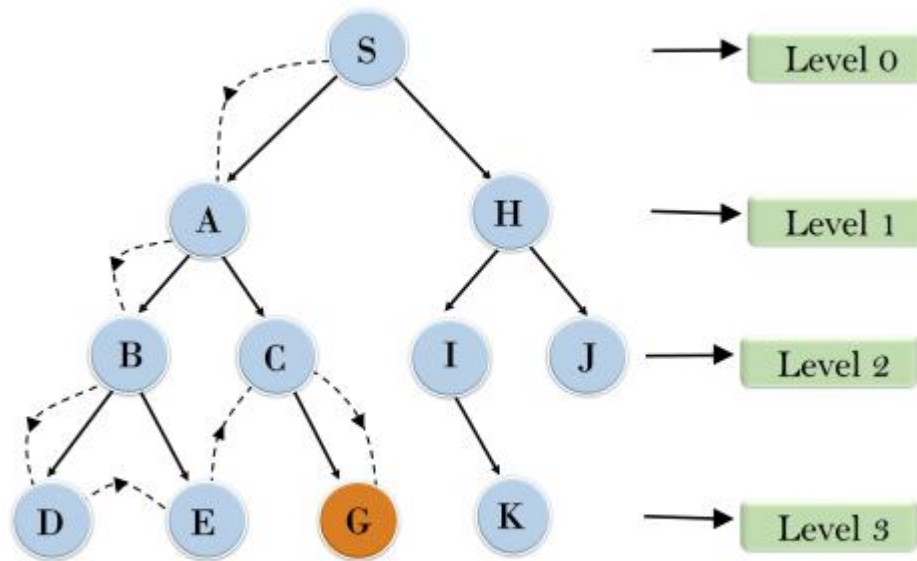
Example:

In the below search tree, we have shown the flow of the depth-first Search, and it will follow the order:

Root node--->Left node ----> Right Node.

It will start searching from root node S and traverse A, then B, then D, and E; after traversing E, it will backtrack the tree as E has no other successor, and the goal node is still not found. After backtracking, it will traverse node C and then G, and here, it will terminate as it found the goal node.

Depth First Search



Completeness: The DFS search algorithm is complete within finite state space as it will expand every Node within a limited search tree.

Time Complexity: The time complexity of DFS will be equivalent to the Node traversed by the algorithm. It is given by:

$$T(n) = 1 + n^2 + n^3 + \dots + n^m = O(n^m)$$

Where m = maximum depth of any node, and this can be much larger than d (Shallowest solution depth)

Space Complexity: The DFS algorithm needs to store only a single path from the root node. Hence, the space complexity of DFS is equivalent to the size of the fringe set, which is **$O(bm)$** .

Optimal: The DFS search algorithm is non-optimal, as it may generate a large number of steps or high Cost to reach the goal node.

3. Uniform-cost Search Algorithm:

Uniform-cost Search is a searching algorithm used for traversing a weighted tree or graph. This algorithm is used when a different cost is available for each edge. The primary goal of the uniform-cost Search is to find a path to the goal node that has the lowest cumulative Cost. Uniform-cost Search expands nodes according to their path costs from the root node.

It can be used to solve any graph/tree where the optimal Cost is in demand. The priority queue implements a uniform-cost search algorithm. It gives maximum priority to the lowest cumulative Cost. A uniform cost search is equivalent to the BFS algorithm if the path cost of all edges is the same.

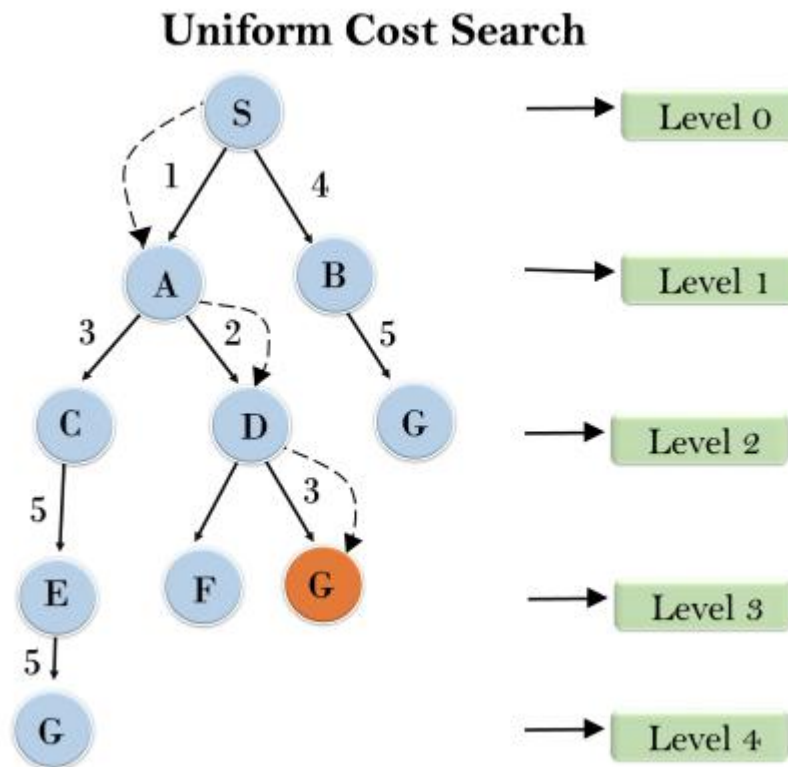
Advantages:

- A uniform cost search is optimal because, at every state, the path with the least cost is chosen.
- It is efficient when the edge weights are small, as it explores the paths in an order that ensures that the shortest path is found early.
- It's a fundamental search method that is not overly complex that makes it accessible for many users.
- It is a type of comprehensive algorithm that will find a solution if one exists. This means the algorithm is complete and ensures that it can locate a solution whenever a viable one is available. The algorithm covers all the necessary steps to arrive at a resolution.

Disadvantages:

- It does not care about the number of steps involved in searching and is only concerned about path cost. Due to this, this algorithm may be stuck in an infinite loop.
- When in operation, UCS shall know all the edge weights to start the Search.
- This Search holds constant the list of the nodes that it has already discovered in a priority queue. Such is a much weightier thing if you have a large graph. The algorithm allocates the memory by storing the path sequence of prioritizes, which can be memory-intensive as the graph gets larger.
- With the help of a Uniform cost search, we can end up with the problem if the graph has edge cycles with a smaller cost than that of the shortest path.
- The Uniform cost search will keep deploying a priority queue so that the paths explored can be stored in any case as the graph size can be even bigger, which can eventually result in too much memory being used.

Example:

**Completeness:**

Uniform-cost Search is complete; if there is a solution, UCS will find it.

Time Complexity:

Let C^* be the Cost of the optimal solution, and ϵ is each step to get closer to the goal node. Then the number of steps is $= C^*/\epsilon + 1$. Here, we have taken $+1$, as we start from state 0 and end with C^*/ϵ .

Hence, the worst-case time complexity of the Uniform-cost search is $(b^{1 + \lceil C^*/\epsilon \rceil})$.

Space Complexity:

The same logic is for space complexity, so the worst-case space complexity of Uniform-cost search is $O(b^{1 + \lceil C^*/\epsilon \rceil})$.

Optimal:

Uniform-cost Search is always optimal as it only selects a path with the lowest path cost.

4. Iterative Deepening Depth-first Search (IDDFS):

The iterative deepening algorithm is a combination of DFS and BFS algorithms. This search algorithm finds out the best depth limit and does it by gradually increasing the limit until a goal is found.

This algorithm performs a depth-first Search up to a certain "depth limit", and it keeps increasing the depth limit after each iteration until the goal node is found.

This Search algorithm combines the benefits of Breadth-first search's fast Search and depth-first Search's memory efficiency.

The iterative search algorithm is useful for uninformed Search when the search space is large, and the depth of the goal node is unknown.

Steps for Iterative deepening depth-first search algorithm:

- Set the depth limit to 0.
- Perform DFS to the depth limit.
- If the goal state is found, return it.
- If the goal state is not found and the maximum depth has not been reached, increment the depth limit and repeat steps 2-4.
- If the goal state is not found and the maximum depth has been reached, terminate the search and return failure.

Advantages:

- It combines the benefits of BFS and DFS search algorithms in terms of fast search and memory efficiency.
- It is a type of straightforward which is used to put into practice since it builds upon the conventional depth-first search algorithm.
- It is a type of search algorithm that provides guarantees to find the optimal solution as long as the Cost of each edge in the search space is the same.
- It is a type of complete algorithm, and the meaning of this is it will always find a solution if one exists.
- The Iterative Deepening Depth-First Search (IDDFS) algorithm uses less memory compared to Breadth-First Search (BFS) because it only stores the current path in memory rather than the entire search tree.

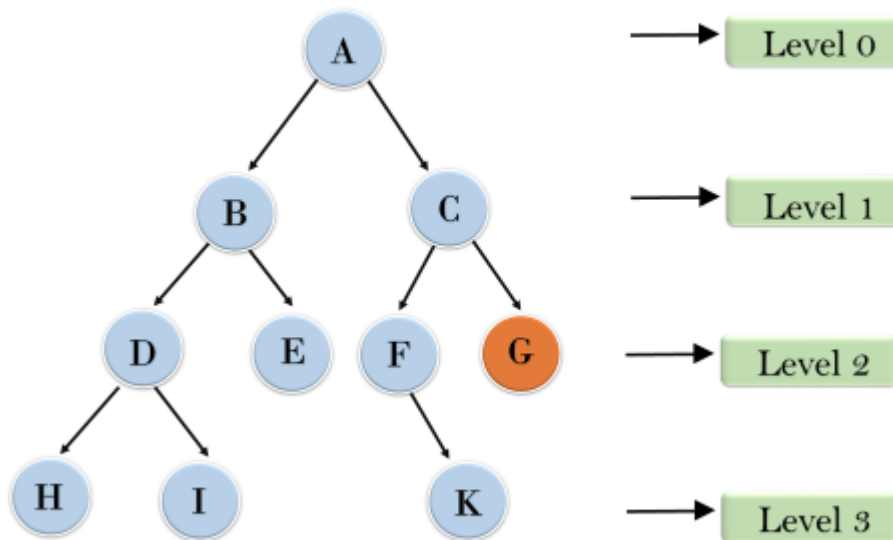
Disadvantages:

The main drawback of IDDFS is that it repeats all the work of the previous phase.

Example:

The following tree structure shows the iterative deepening depth-first Search. The IDDFS algorithm performs various iterations until it cannot find the goal node. The iteration performed by the algorithm is given as:

Iterative deepening depth first search



1'st Iteration-----> A

2'nd Iteration-----> A, B, C

3'rd Iteration----->A, B, D, E, C, F, G

4'th Iteration----->A, B, D, H, I, E, C, F, K, G

In the fourth iteration, the algorithm will find the goal node.

Completeness:

This algorithm is complete if the branching factor is finite.

Time Complexity:

Let's suppose b is the branching factor, depth is d , and then the worst-case time complexity is $O(b^d)$.

Space Complexity:

The space complexity of IDDFS will be $O(bd)$.

Optimal:

IDDFS algorithm is optimal if path cost is a non-decreasing function of the depth of the Node.

5. Bidirectional Search Algorithm:

A bidirectional search algorithm runs two simultaneous searches, one from the initial state called forward-search and the other from the goal node called backward-search, to find the goal node. Bidirectional Search replaces one single search graph with two small subgraphs in which one starts the Search from an initial vertex and the other starts from a goal vertex. The Search stops when these two graphs intersect each other.

Bidirectional Search can use search techniques such as BFS, DFS, DLS, etc.

Advantages:

- Bidirectional Search is fast.
- Bidirectional Search requires less memory.
- The graph can be extremely helpful when it is very large, and there is no way to make it smaller. In such cases, using this tool becomes particularly useful.
- The Cost of expanding nodes can be high in certain cases. In such scenarios, using this approach can help reduce the number of nodes that need to be expanded.

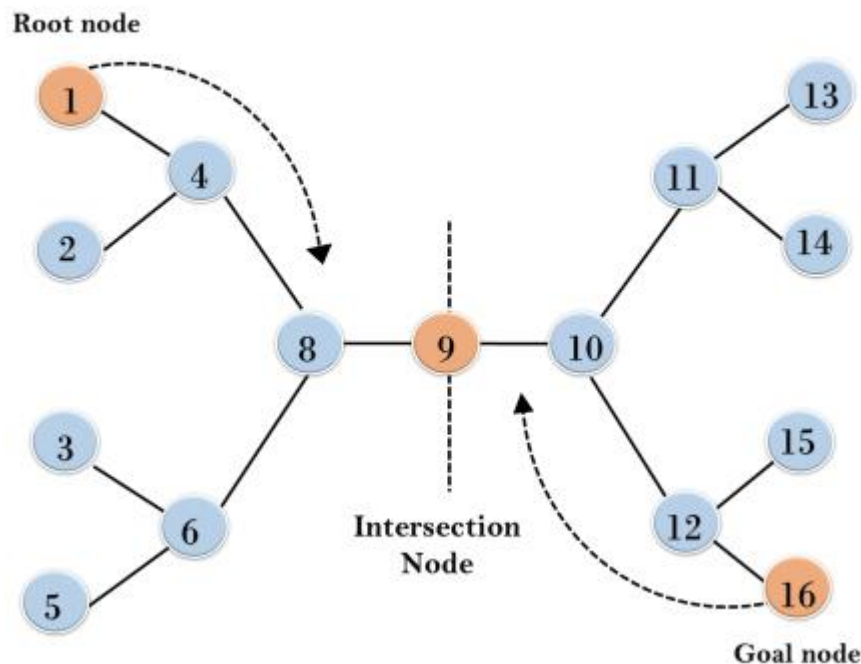
Disadvantages:

- Implementation of the bidirectional search tree is difficult.
- In bidirectional Search, one should know the goal state in advance.
- Finding an efficient way to check if a match exists between search trees can be tricky, which can increase the time it takes to complete the task.

Example:

In the below search tree, the bidirectional search algorithm is applied. This algorithm divides one graph/tree into two sub-graphs. It starts traversing from node 1 in the forward direction and starts from goal node 16 in the backward direction. The algorithm terminates at node 9, where two searches meet.

Bidirectional Search



Completeness: Bidirectional Search is complete if we use BFS in both searches.

Time Complexity: The time complexity of bidirectional Search using BFS is $O(b^d)$.

Space Complexity: The space complexity of bidirectional Search is $O(b^d)$.

Optimal: Bidirectional Search is Optimal.

Applications of Uninformed Search Algorithms

Blind search algorithms, also referred to as uninformed search algorithms, have quite a few applications in various domains. Even though they look and smell simple and lack typical domain-specific heuristics, these algorithms are remarkably powerful for solving fundamental problems in Artificial Intelligence, network routing, puzzle solving, and web crawling.

- **Pathfinding in Games:** This means that characters or agents must traverse through virtual environments in games. For instance, in the type of games in which you must solve mazes, algorithms like Breadth First Search (BFS) and Depth First Search (DFS) are meant for finding paths between different points. Such methods are simple but effective in deterministic environments where the map is completely known.

- **Robotics:** For uninformed Search in robotics, autonomous systems can use it to explore an environment. For example, Uniform Cost Search (UCS) is a strategy that a robot performing consecutive moves through a warehouse grid might use to find the path of the shortest distance to a target location.
- **Planning:** Planning with AI is a task that involves finding a sequence of actions to realize a desired goal. With tasks such as scheduling and logistics and making simple decisions in constrained environments, the Depth First Search (DFS) and Iterative Deepening Depth First Search (IDDFS) are commonly used.

Network Routing

- **Shortest Path Problems:** Uniform-Cost Search (UCS) is the perfect algorithm for finding the cheapest path if we have two network nodes. It is applied to the routing of data packets in computer networks, to find optimal transportation routes, etc., and to manage power grids.
- **Connection Troubleshooting:** It helps in tracing a path to debug a network to trace all nodes reachable from a fixed source using BFS.

Puzzle Solving

- **Sliding Puzzles:** Sliding tile puzzles like the 8 puzzle or 15 puzzles are puzzles where tiles need to be arranged into some specific order and solved by finding a sequence of moves to do it. BFS guarantees that the solution will be optimal, and DFS can quickly find any solution.
- **N-Queens Problem:** The N-Queens Problem is to place exactly N queens on an N x N chessboard with the constraint that no two queens can attack each other. Systematic exploration of board configurations is done with Depth Limited Search and IDDFS.
- **Sudoku Solving:** While an uninformed search is not the most efficient approach for solving a Sudoku problem, it can help us complete enumerations of possibilities and check for solutions.

Web Crawling

- **Efficient Traversal:** In particular, BFS is used when web crawling pages to make sure all reachable pages from a starting URL are explored. This method is particularly useful when used to index/search on search engines or during exhaustive Search through connected web pages.

- **Depth Management:** To control the level of exploration in large web graphs, Depth Limited Search is used to limit the number of links followed so systems are not overloaded.

Strengths of Uninformed Search Algorithms

- **Universality:** All of it is given in terms of uninformed search algorithms that can be applied to any graph or tree problem. The obvious general-purpose nature of them makes them suitable for general-purpose problem-solving without requiring such problem-specific heuristics.
- **Simplicity:** They are easy to formulate and even easier to implement. Larger popular examples like BFS and DFS have very little conceptual content that allows you to understand them without much work, making them ideal as both building blocks for other, more sophisticated techniques as well as educational tools.
- **Guaranteed Solution (When Applicable):** BFS is an algorithm that is complete; it guarantees that a solution is found if one exists, and, in the case of the Search, space is finite. A major advantage of this reliability is in the case where a solution must be ensured, which is not always true.
- **Optimality (In Specific Cases):** Unlike BFS, to find the shortest path to the goal in unweighted graphs, BFS guarantees to find the shortest path. In cases where the path cost is uniform or irrelevant, this property provides a benefit.

Weaknesses of Uninformed Search Algorithms

- **High Resource Consumption (Time and Space):** In the worst case, such uninformed search algorithms often explore vast regions of the search space and, therefore, have exponential time complexity. A simple example is that BFS needs to maintain a queue of all frontier nodes, and this will overspend memory in large search spaces.
- **Inefficiency in Large or Complex Search Spaces:** Algorithms such as DFS are likely to fall into infinite loops in environments with many nodes or having deep solutions. This is because they use no heuristic information for steering, and so they have biased behavior in exploring irrelevant paths.
- **Lack of Domain Knowledge Utilisation:** Without taking such problem-specific insights into account, uninformed algorithms tend to result in an order of magnitude increase in computational effort. The

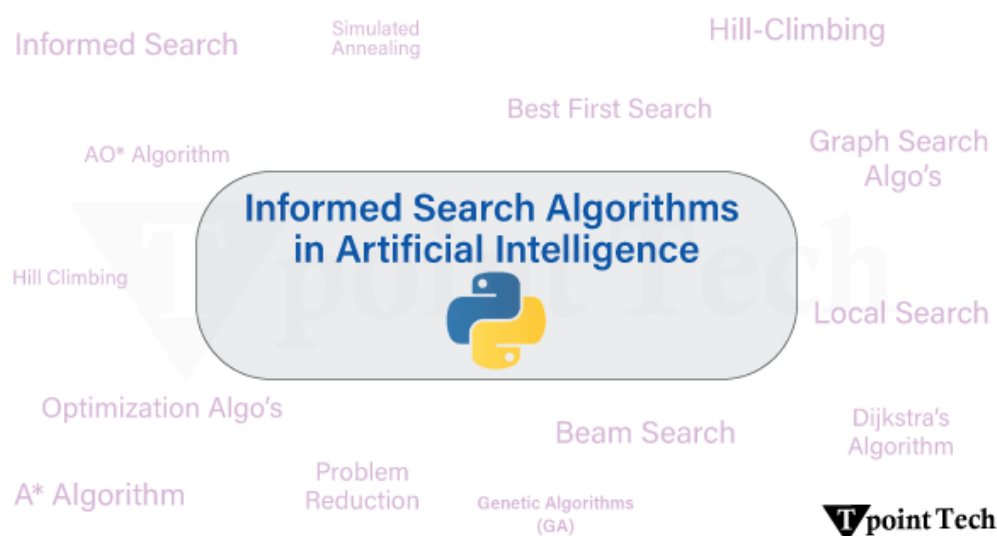
algorithms, such as A*, use heuristics to guide the Search more efficiently, while in contrast to them, the algorithm is not informed.

- **Not Always Optimal:** For example, DFS does not ensure a shortest path, as it is concerned with depth before minimizing Cost. For weighted machines, BFS is not optimal either.

4.2 Informed Search Algorithms

Informed Search Algorithm is one of the most essential components of Artificial Intelligence. The search techniques in informed search operate in the most efficient and optimal way. The Informed search algorithms improve the efficiency of the searching process by using the knowledge of specific domains.

The Informed Search Algorithm can also be termed the Heuristic Search. Heuristic Search means searching for ways, paths, options, or procedures that can be determined as the most efficient, productive, and cost-saving choice to reach the pre-determined goal.



Key Characteristics of Informed Search Algorithms

Heuristic Function: The Heuristic function is used by the Informed Search Algorithm, also known as the Heuristic Search. To assist in getting to the goal node, it provides the shortest path and minimal cost.

Efficiency: Talking about efficiency, the Informed search is more efficient and explores the solutions quickly than the other searches, for example, Uninformed Searches.

Optimality: The Information Search Algorithm produces the most optimal solution possible if it exists. An algorithm is generally considered complete if

a solution is guaranteed, and an optimal solution if it's available. For example, A* Search Algorithm is a complete algorithm.

Heuristic Function

The Heuristic Function is a blueprint that uses the power of making the best guess in the problem to find out the cost or distance to the goal state. The Heuristic function is used by the heuristic search to calculate the best distance or cost quite accurately.

It only focuses on the shortest or least cost path, and may avoid the better paths.

Just like how you could take the shortest but bad road to reach somewhere instead of the good road just to save fuel.

For example, in the game of chess, the player who starts first usually has an upper hand in winning the game. In terms of heuristic function, the person who controls the centre of the chessboard has more chances of winning.

Types of Informed Search

There are several types of Informed Search in Artificial Intelligence. We will discuss some of them in detail to understand the topic in depth.

1. Greedy Best-First Search

Greedy Best-First Search in AI is a graph traversal algorithm designed to find the shortest path or solve problems with multiple possible solutions. It is a [heuristic search algorithm](#), means it uses a heuristic function to guide the search process by estimating how close a path is to the goal.

Unlike search methods that don't have prior knowledge, it uses heuristic rules or estimates that gives direction for search. It is popular because of its simplicity and efficiency in certain situations. This makes it an informed search algorithm because it uses problem-specific knowledge to decide the next step, focusing on the most promising path without revisiting previous choices.

How Does Greedy Best-First Search Work?

Greedy Best-First Search algorithm operates by exploring nodes in a graph by always choosing the one that appears closest to the goal. The algorithm evaluates the nodes based on a heuristic function which estimates the cost from the current node to the goal. It's called "greedy" because it selects the node that seems to be the best choice at the moment, focusing only on immediate progress.

The algorithm does not backtrack or reconsider nodes that seem less optimal making it fast but potentially incomplete if the path it chooses does not lead to a solution.

Let's see step-by-step how Greedy Best-First Search algorithm works:

1. **Initialization:** We start at the initial node and add it to a priority queue.
2. **Expand Nodes:** The algorithm evaluates all neighboring nodes, assigning each one a value based on the heuristic function (representing the estimated distance to the goal).
3. **Select the Best Node:** From the priority queue, the node with the lowest heuristic value is selected. This node is considered the most promising.
4. **Goal Check:** If the selected node is the goal, the search terminates. If not, the algorithm continues exploring.
5. **Repeat:** Steps 2-4 are repeated until the goal is found or the search space is exhausted.

Role of Heuristics in Greedy Best-First Search (GBFS)

Greedy Best-First Search highly depends on the heuristic function used. A heuristic is a function that estimates the cost to reach the goal from the current node. A good heuristic leads to fast and efficient searching while a poor heuristic can cause inefficiency or even failure.

Common heuristics include:

- **Euclidean Distance:** Straight-line distance between the current node and the goal, used in pathfinding problems.
- **Manhattan Distance:** The sum of absolute differences between the coordinates of two points, often applied in grid-based environments.

The choice of heuristic influences how the algorithm behaves and whether it can find an optimal solution efficiently.

Advantages of Greedy Best-First Search

Greedy Best-First Search offers various advantages that make it a popular choice in various AI applications:

1. **Speed:** It is generally faster than uninformed search algorithms like Breadth-First Search because it uses heuristic information.

2. **Simplicity:** The algorithm is simple to implement and understand, making it a preferred choice in cases where speed is more important than optimality.
3. **Resource-Efficient:** Since it focuses on the path that seems most promising, it may require less memory compared to algorithms like A*.

Limitations of Greedy Best-First Search

Despite its advantages, Greedy Best-First Search has several limitations that can impact its effectiveness.

1. **Incomplete:** It is not guaranteed to find a solution especially if it gets stuck in a local optimum or reaches a dead-end. Since there is no backtracking, it may overlook valid paths that require revisiting nodes to get a better result.
2. **Non-Optimal:** Unlike A*, it does not guarantee an optimal solution. Its greedy nature makes it focus on the nearest node which may not always lead to the best path. This can result in suboptimal solutions especially in complex problem spaces.
3. **Heuristic Dependency:** The efficiency and success of GBFS depends on the quality of the heuristic. A bad heuristic could mislead the search, leading to inefficient exploration or even failure to find a path.

Applications of Greedy Best-First Search in AI

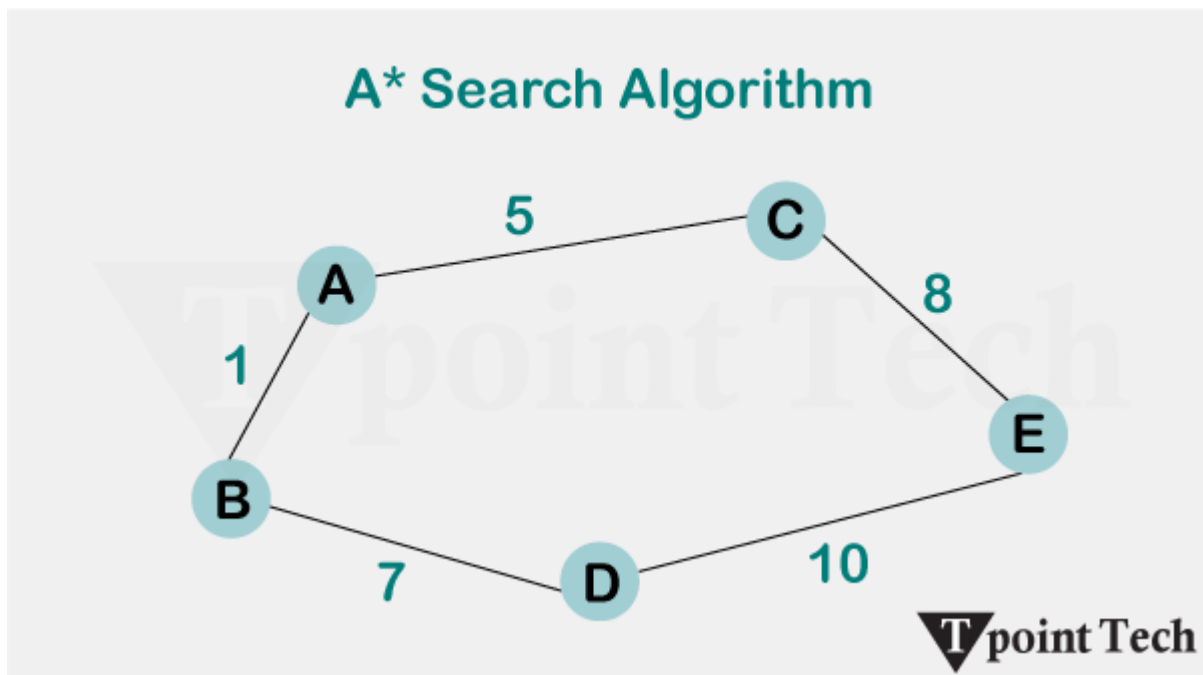
It has various practical applications across various AI and computer science fields:

1. **Pathfinding:** It is used in map navigation systems and video game AI for efficient pathfinding from one point to another.
2. **Puzzle Solving:** In puzzles such as the 8-puzzle or 15-puzzle, it is often used to find the quickest way to reach a goal state, i.e arranging tiles in the correct order.
3. **Robot Navigation:** Autonomous robots use GBFS for navigation in unknown or dynamic environments as the heuristic helps the robot choose paths that appear to bring it closest to the goal efficiently.
4. **Artificial Intelligence Planning:** It is used in various AI planning tasks such as automated problem-solving where there are multiple potential solutions and a fast solution is more important than the most optimal one.

2.A* Search Algorithm

A* (pronounced "A-star") is a powerful graph traversal and pathfinding algorithm widely used in artificial intelligence and computer science. It is mainly used to find the shortest path between two nodes in a graph, given the estimated cost of getting from the current node to the destination node. The main advantage of the algorithm is its ability to provide an optimal path by exploring the graph in a more informed way compared to traditional search algorithms such as Dijkstra's algorithm.

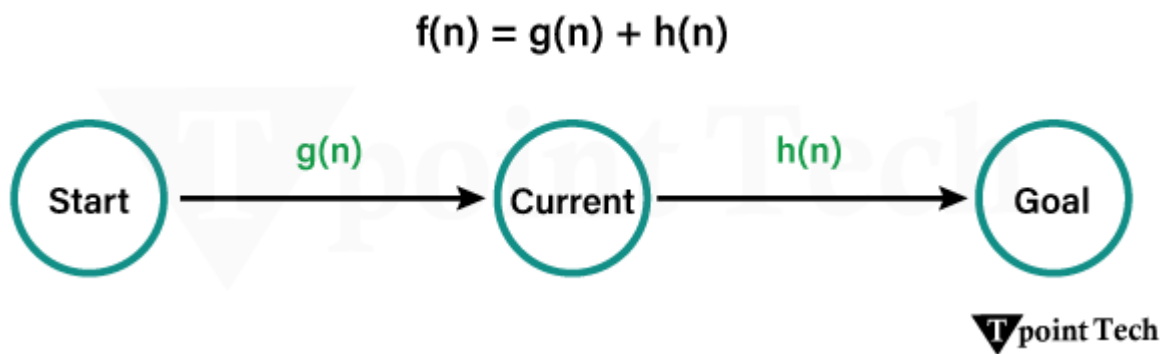
Algorithm A* combines the advantages of two other search algorithms: Dijkstra's algorithm and Greedy Best-First Search. Like Dijkstra's algorithm, A* ensures that the path found is as short as possible but does so more efficiently by directing its search through a heuristic similar to Greedy Best-First Search. A heuristic function, denoted $h(n)$, estimates the cost of getting from any given node n to the destination node.



Formula of A* Search Algorithm

As we know, A* Search Algorithm is used to identify the most cost-effective and the shortest path from one node to another node in a graph.

Let's take a look at the formula that we use to find the total cost of exploring the shortest path from one node to another node:



$$f(n) = g(n) + h(n)$$

Where,

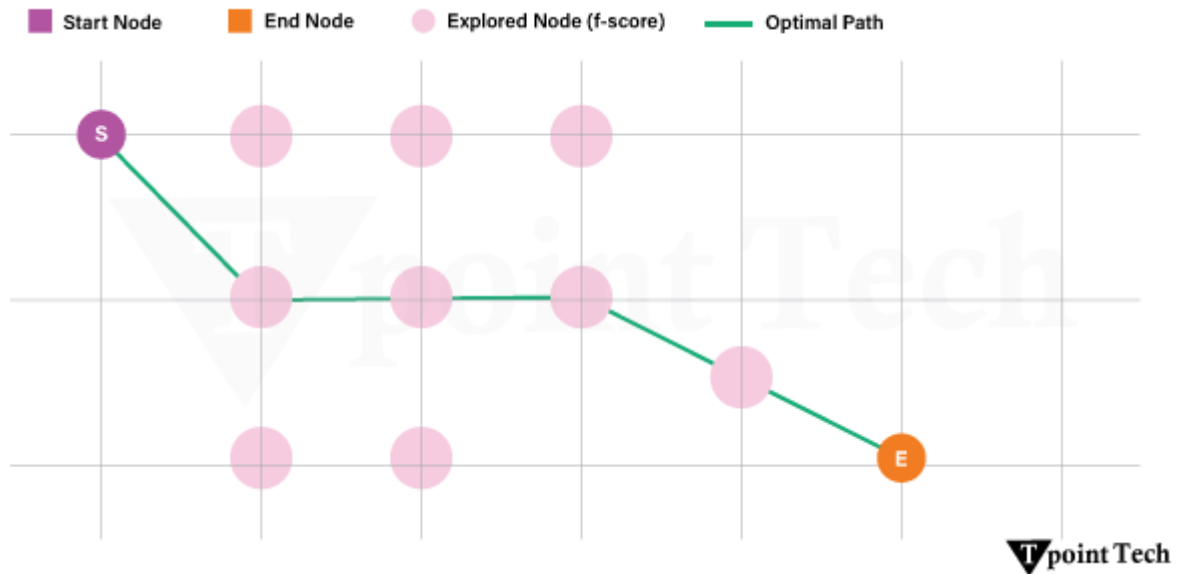
- **f(n):** The total cost that occurs from the initial node to the final state via the current node n is represented by f(n).
- **g(n):** the actual cost to get from the initial node to node n. It represents the sum of the costs of node n outgoing edges.
- **h(n):** Heuristic cost (also known as "estimation cost") from node n to destination node n. This problem-specific heuristic function must be acceptable, meaning it never overestimates the actual cost of achieving the goal. The evaluation function of node n is defined as $f(n) = g(n) + h(n)$.

Algorithm A* selects the nodes to be explored based on the lowest value of f(n), preferring the nodes with the lowest estimated total cost to reach the goal. The A* algorithm works:

Key components

Let's understand the key components of the A* Search Algorithm:

- **Nodes:** Nodes are the points in the graph(where connections meet)
- **Edges:** These are connections between the Nodes
- **Path Cost:** The Cost of traversing from one node to another node.
- **Heuristic:** Estimated cost of traversing to the final or goal node from the current node
- **Search Space:** It is the group of all possible paths that can be explored



How does the A* Search Algorithm work?

A* search blends the principles of Dijkstra's algorithm with those of the Greedy Best-First Search. This gives the A* search algorithms the best of both worlds.

Let's take a quick look at the explanation of both algorithms:

- **Dijkstra's Algorithm:** Dijkstra's Algorithm helps to find the minimum-cost paths from a given source node to every other node in a graph.
- **Greedy Best-First Search Algorithm:** In the [Greedy Best-First Search](#) algorithm, the node estimated to be nearest to the goal state is chosen for expansion.

The best of both worlds combines to form A* search, which makes it more efficient and accurate than they were individually.

Step-by-Step Implementation of the A* Algorithm

- **Initialization:** In the first step, the initial nodes are placed in the priority queue. The priority queue consists of all nodes that are pending evaluation. It is also known as the open list.
- **Evaluation:** In the second step, the node with the lowest $f(n)$ value is selected from the queue to be processed next.
- **Expansion:** The node with the lowest $f(n)$ value that was chosen in the previous evaluation step is compared with its neighbours.

- **Update:** After the comparison, if a neighbouring node has a lower $f(n)$ value than the one previously recorded, then it is updated and added to the open list.
- **Goal Check:** In this step, the goal check is done to determine whether the goal node is achieved. This can also be confirmed if the open list is vacant, which tells that there is nothing more for further traversal.
- **Reconstruction:** In the final step, the path is reconstructed by backtracking or tracing the steps from the final or goal node or the initial node.

Understanding Heuristic Search $h(n)$

Heuristic function $h(n)$ plays a crucial role in the A* search algorithm as it helps to find the estimated cost of traversing to the final or goal node from the current node.

There are several types of Heuristic search. Let's understand them in detail:

Manhattan Distance: The Manhattan Distance is calculated as the sum of the net differences between the current position and the goal position of x and y coordinates.

Sounds confusing, right? Let's understand this with a formula that you might have learned in geometry.

Formula:

$$h(n) = |x_{\text{current}} - x_{\text{goal}}| + |y_{\text{current}} - y_{\text{goal}}|$$

Where,

$h(n)$ = Heuristic Function

Euclidean Function: The Euclidean distance is calculated as the square root of the sum of the squares of the differences between the x and y coordinates of the current position and the goal position.

Let's see the above statement in the form of a formula.

Formula:

$$h(n) = \sqrt{(x_{\text{current}} - x_{\text{goal}})^2 + (y_{\text{current}} - y_{\text{goal}})^2}$$

Diagonal Distance: The Diagonal is the distance that is measured when the number of movements of directions possible is 8. For example, a Robot vacuum cleaner or a King in the game of chess.

Let's see the above statement in the form of a formula.

Formula:

$$h(n) = \max(|x_{\text{current}} - x_{\text{goal}}|, |y_{\text{current}} - y_{\text{goal}}|)$$

Advantages of A* Search Algorithm in Artificial Intelligence

The A* search algorithm offers several advantages in artificial intelligence and problem-solving scenarios:

1. **Optimal solution:** A* ensures finding the optimal (shortest) path from the start node to the destination node in the weighted graph given an acceptable heuristic function. This optimality is a decisive advantage in many applications where finding the shortest path is essential.
2. **Completeness:** If a solution exists, A* will find it, provided the graph does not have an infinite cost. This completeness property ensures that A* can take advantage of a solution if it exists.
3. **Efficiency:** A* is efficient if an efficient and acceptable heuristic function is used. Heuristics guide the search to a goal by focusing on promising paths and avoiding unnecessary exploration, making A* more efficient than non-aware search algorithms such as breadth-first search or depth-first search.
4. **Versatility:** A* is widely applicable to various problem areas, including wayfinding, route planning, robotics, game development, and more. A* can be used to find optimal solutions efficiently as long as a meaningful heuristic can be defined.
5. **Optimized search:** A* maintains a priority order to select the nodes with the minor $f(n)$ value ($g(n)$ and $h(n)$) for expansion. This allows it to explore promising paths first, which reduces the search space and leads to faster convergence.
6. **Memory efficiency:** Unlike some other search algorithms, such as breadth-first search, A* stores only a limited number of nodes in the priority queue, which makes it memory efficient, especially for large graphs.
7. **Tunable Heuristics:** A*'s performance can be fine-tuned by selecting different heuristic functions. More educated heuristics can lead to faster convergence and less expanded nodes.
8. **Extensively researched:** A* is a well-established algorithm with decades of research and practical applications. Many optimizations and variations have been developed, making it a reliable and well-understood troubleshooting tool.

9. **Web search:** A* can be used for web-based path search, where the algorithm constantly updates the path according to changes in the environment or the appearance of new It enables real-time decision-making in dynamic scenarios.

Disadvantages of A* Search Algorithm

Although the A* (letter A) search algorithm is a widely used and powerful technique for solving AI pathfinding and graph traversal problems, it has disadvantages and limitations. Here are some of the main disadvantages of the search algorithm:

1. **Heuristic accuracy:** The performance of the A* algorithm depends heavily on the accuracy of the heuristic function used to estimate the cost from the current node to the If the heuristic is unacceptable (never overestimates the actual cost) or inconsistent (satisfies the triangle inequality), A* may not find an optimal path or may explore more nodes than necessary, affecting its efficiency and accuracy.
2. **Memory usage:** A* requires that all visited nodes be kept in memory to keep track of explored paths. Memory usage can sometimes become a significant issue, especially when dealing with an ample search space or limited memory resources.
3. **Time complexity:** Although A* is generally efficient, its time complexity can be a concern for vast search spaces or graphs. In the worst case, A* can take exponentially longer to find the optimal path if the heuristic is inappropriate for the problem.
4. **Bottleneck at the destination:** In specific scenarios, the A* algorithm needs to explore nodes far from the destination before finally reaching the destination region. This the problem occurs when the heuristic needs to direct the search to the goal early effectively.
5. **Cost Binding:** A* faces difficulties when multiple nodes have the same f-value (the sum of the actual cost and the heuristic cost). The strategy used can affect the optimality and efficiency of the discovered path. If not handled correctly, it can lead to unnecessary nodes being explored and slow down the algorithm.
6. **Complexity in dynamic environments:** In dynamic environments where the cost of edges or nodes may change during the search, A* may not be suitable because it does not adapt well to such changes. Reformulation from scratch can be computationally expensive, and D* (Dynamic A*) algorithms were designed to solve this

7. **Perfection in infinite space** : A* may not find a solution in infinite state space. In such cases, it can run indefinitely, exploring an ever-increasing number of nodes without finding a solution. Despite these shortcomings, A* is still a robust and widely used algorithm because it can effectively find optimal paths in many practical situations if the heuristic function is well-designed and the search space is manageable. Various variations and variants of A* have been proposed to alleviate some of its limitations.

2. Hill Climbing Algorithm

Hill climbing algorithm is a local search algorithm that continuously moves in the direction of increasing elevation/value to find the peak of the mountain or the best solution to the problem. It terminates when it reaches a peak value where no neighbor has a higher value.

It is a technique which is used for optimizing the mathematical problems. One of the widely discussed examples of Hill climbing algorithm is Traveling-salesman Problem in which we need to minimize the distance traveled by the salesman.

It is also called greedy local search as it only looks to its good immediate neighbor state and not beyond that. A node of hill climbing algorithm has two components which are state and value. Hill Climbing is mostly used when a good heuristic is available. In this algorithm, we don't need to maintain and handle the search tree or graph, as it only keeps a single current state.

Advantages of Hill climb algorithm:

The merits of the Hill Climbing algorithm are given below.

- The first part of the paper is based on Hill's diagrams that can easily put things together, ideal for complex optimization problem. A hill climbing algorithm is first invention of hill climbers.
- It uses much less RAM for the current problem state and the solutions located around it than comparing the algorithm to a tree search method, which will require inspecting the entire tree. Consequently, this reduces the total memory resources to be used. Space is what matters; solutions should occupy a convenient area to consume as little memory as possible.
- When it comes to the acceleration of the hill up, most of the time it brings a closure in the local maximum straight away. This is the route

if having quickly getting a quick solution, outshining acquiring a global maximum, is an incentive.

Disadvantages of Hill Climbing Algorithm

- Concerning hill climbing, it seems that some solutions do not find the optimum point and remain stuck at a local peak, particularly where the optimization needs to be done in complex environments with many objective functions.
- It is also superficial because it just seeks for the surrounding solution and does not get farther than that. It could be on a wrong course which is based on a locally optimal solution, and consequently Godbole needs to move far away from current position in the first place.
- It is highly likely that the end result will largely depend on the initial setup and state, with a precedent of it being the most sensitive factor. It implies that in this case, time is the perimeter of the sphere within which people develop their skills dynamically, determining their success.

Features of Hill Climbing:

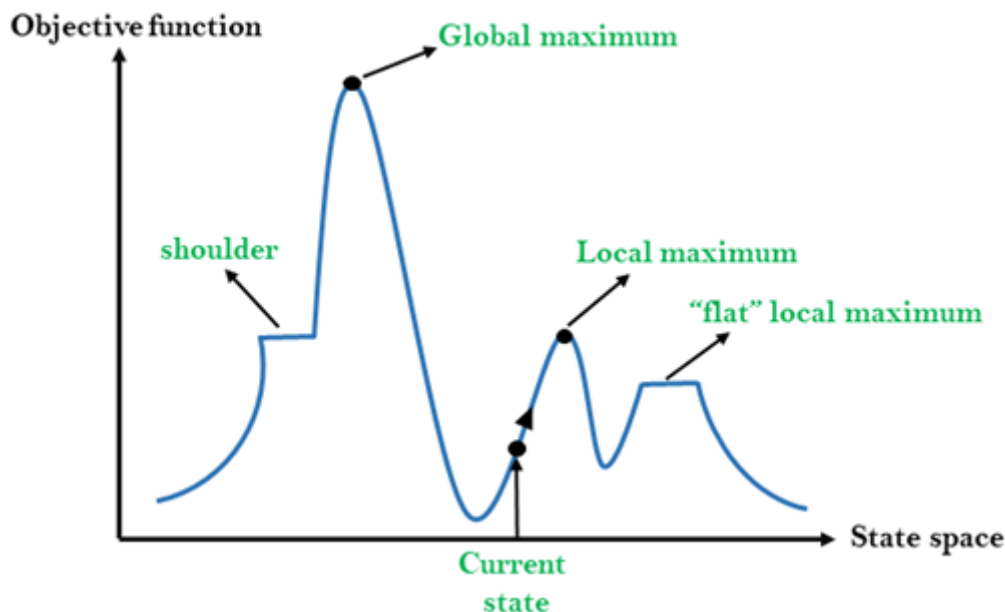
The following are some main features of the Hill Climbing Algorithm:

- **Generate and Test variant:** Hill Climbing is a variant of the Generate and Test method. The Generate and Test method produces feedback that helps to decide which direction to move in the search space.
- **Greedy approach:** Hill-climbing algorithm search moves in the direction that optimizes the cost.
- **No backtracking:** It does not backtrack the search space, as it does not remember the previous states.
- **Deterministic Nature:** Hill Climbing is a deterministic optimization algorithm, which means that given the same initial conditions and the same problem, it will always produce the same result. There is no randomness or uncertainty in its operation.
- **Local Neighborhood:** Hill Climbing is a technique that operates within a small area around the current solution. It explores solutions that are closely related to the current state by making small, gradual changes. This approach allows it to find a solution that is better than the current one, although it may not be the global optimum.

State-space Diagram for Hill Climbing:

The state-space landscape is a graphical representation of the hill-climbing algorithm which is showing a graph between various states of algorithm and Objective function/Cost.

On Y-axis we have taken the function which can be an objective function or cost function, and state-space on the x-axis. If the function on Y-axis is cost then, the goal of search is to find the global minimum and local minimum. If the function of Y-axis is Objective function, then the goal of the search is to find the global maximum and local maximum.



Different regions in the state space landscape:

- **Local Maximum:** Local maximum is a state which is better than its neighbor states, but there is also another state which is higher than it.
- **Global Maximum:** Global maximum is the best possible state of state space landscape. It has the highest value of objective function.
- **Current State:** It is a state in a landscape diagram where an agent is currently present.
- **Flat local maximum:** It is a flat space in the landscape where all the neighboring states of the current state have the same value.
- **Shoulder:** It is a plateau region that has an uphill edge.

Types of Hill Climbing Algorithm:

- Simple hill Climbing:
- Steepest-Ascent hill-climbing:

- Stochastic hill Climbing:

1. Simple Hill Climbing:

Simple hill climbing is the simplest way to implement a hill-climbing algorithm. It only evaluates the neighbor node state at a time and selects the first one that optimizes the current cost, and sets it as the current state. It only checks its one successor state, and if it finds a better state than the current state, then it moves, else it remains in the same state. This algorithm has the following features:

- Less time-consuming
- A less optimal solution, and the solution is not guaranteed

Algorithm for Simple Hill Climbing:

- **Step 1:** Evaluate the initial state. If it is the goal state, then return success and Stop.
- **Step 2:** Loop until a solution is found or there is no new operator left to apply.
- **Step 3:** Select and apply an operator to the current state.
- **Step 4:** Check the new state:
 - If it is the goal state, then return success and quit.
 - Else if it is better than the current state, then assign the new state as the current state.
 - Else if not better than the current state, then return to step 2.

Simulated Annealing Search

Simulated Annealing Search is a **local search algorithm** used to find a good solution for difficult optimization problems.

It is mainly used when the search space is very large and simple methods like **hill climbing** may get stuck.

1. Simple Meaning

Simulated annealing is a search method that sometimes accepts a worse solution temporarily to escape from a local optimum and find a better solution later.

In simple words:

It does not always move only toward better states. Sometimes it takes a bad move so that it can avoid getting stuck.

2. Why is it called Simulated Annealing?

The idea comes from **annealing in metallurgy**.

In metallurgy:

1. A metal is heated to a high temperature.
2. Then it is cooled slowly.
3. This process makes the metal strong and stable.

In AI:

Metallurgy	AI Search
High temperature	More random search
Slow cooling	Gradually reduce randomness
Stable metal	Good final solution

3. Why Do We Need Simulated Annealing?

Hill climbing always chooses a better neighboring state.

But hill climbing can get stuck in:

Problem	Meaning
Local maximum	Better than nearby states, but not best overall
Plateau	Flat region where all neighboring states look same
Ridge	Narrow path where improvement is difficult

Simulated annealing solves this by sometimes accepting a worse move.

4. Simple Mountain Example

Imagine you are climbing mountains in the dark.

Your goal is to reach the **highest mountain**.

Hill Climbing

UNIT-I

Hill climbing always moves upward.

Problem:

It may reach a small hill and stop there.

Small hill = local maximum

Highest hill = global maximum

Simulated Annealing

Simulated annealing may sometimes move downward.

Why?

Because after moving down, it may later climb a much higher mountain.

So simulated annealing has a better chance of finding the global best solution.

5. Main Idea

Simulated annealing works like this:

If the new state is better → accept it.

If the new state is worse → sometimes accept it.

At the beginning:

Temperature is high → bad moves are accepted more often

Later:

Temperature is low → bad moves are rarely accepted

So the algorithm explores more in the beginning and becomes more careful later.

6. Acceptance Probability

The probability of accepting a worse move is commonly written as:

$$P = e^{-\Delta E/T}$$

A

$$y = Ae^{-kt} = 6e^{-0.6t}$$

Where:

Symbol	Meaning
P	Probability of accepting a worse move
ΔE	Difference between new state and current state
T	Temperature

Simple Meaning

If temperature is high:

Probability is high $\rightarrow$ worse move may be accepted

If temperature is low:

Probability is low $\rightarrow$ worse move is usually rejected

7. Steps of Simulated Annealing Algorithm

1. Start with an initial solution.
2. Set a high temperature.
3. Generate a neighboring solution.
4. If the neighbor is better, accept it.
5. If the neighbor is worse, accept it with some probability.
6. Reduce the temperature slowly.
7. Repeat the process.
8. Stop when temperature becomes very low or stopping condition is reached.

8. Example

Suppose an AI agent is solving a travelling salesman problem.

Goal:

Find the shortest route visiting all cities.

The algorithm starts with one route.

Then it changes the route slightly.

If the new route is shorter, it accepts it.

If the new route is longer, it may still accept it in the beginning.

This helps the algorithm avoid getting stuck in a poor route.

9. Hill Climbing vs Simulated Annealing

Point	Hill Climbing	Simulated Annealing
Accepts better moves	Yes	Yes
Accepts worse moves	No	Sometimes
Can escape local optimum	No	Yes
Uses temperature	No	Yes
Search behavior	Greedy	Controlled random search
Final solution	May be local optimum	Better chance of global optimum

10. Applications

Area	Application
Travelling Salesman Problem	Finding shortest tour
Timetable scheduling	Creating efficient schedules
Robotics	Path optimization
VLSI design	Chip layout optimization
Machine learning	Parameter optimization
Network routing	Finding low-cost routes

11. Advantages

Advantage	Explanation
Escapes local optimum	Accepts worse moves sometimes
Useful for large problems	Works when search space is huge
Flexible	Can be applied to many optimization problems
Better than hill climbing	Does not stop easily at local optimum

12. Disadvantages

Disadvantage	Explanation
Can be slow	Needs many iterations

Disadvantage	Explanation
Depends on cooling schedule	Poor cooling gives poor result
No quick guarantee	May not always find global best quickly
Parameter tuning needed	Temperature and cooling rate must be chosen carefully

Local Search Algorithm

1. Meaning

Local Search in Continuous Spaces is an AI search technique used when the values in a problem are **continuous**, meaning they can take any real-number value.

Examples of continuous values:

2.5, 3.76, 10.45, 0.001, 98.6

In simple words:

Local search in continuous spaces means finding the best solution by making small changes in real-number values.

2. Why is it called Continuous Space?

In some AI problems, the states are separate and countable.

Example:

Move left

Move right

Move up

Move down

This is called **discrete search space**.

But in continuous space, values can be decimal numbers.

Example:

Speed = 45.7 km/hr

Temperature = 36.8°C

Robot angle = 27.5°

Weight = 0.453

So, the number of possible states is very large or infinite.

3. Simple Example

Suppose we want to minimize this function:

$$f(x) = x^2$$

We want to find the value of x where the function value is minimum.

$$x \quad f(x) = x^2$$

5 25

4 16

3 9

2 4

1 1

0 0

The minimum value is:

$$x = 0$$

So the search may move like this:

$$x = 5 \rightarrow x = 4 \rightarrow x = 3 \rightarrow x = 2 \rightarrow x = 1 \rightarrow x = 0$$

This is local search because it improves the current value step by step.

4. Main Idea

Local search starts with one current solution.

Then it checks nearby solutions.

If a nearby solution is better, it moves to that solution.

Start with current solution



Make small change



Check whether it is better



Move to better solution

↓
Repeat

5. Important Terms

Term	Meaning
Current state	Present solution
Neighbor state	Slightly changed solution
Objective function	Function to maximize or minimize
Step size	Amount of change made in each step
Local optimum	Best nearby solution, but not best overall
Global optimum	Best solution in the entire search space

6. Objective Function

An **objective function** is the function that the algorithm tries to optimize.

It may be:

Minimization

Minimize error
Minimize distance
Minimize cost

Maximization

Maximize accuracy
Maximize profit
Maximize performance

In machine learning, the common objective is:

Minimize loss or error

7. Discrete Search vs Continuous Search

Point	Discrete Search	Continuous Search
Values	Fixed/countable values	Real-number values
Example	8-puzzle moves	$x = 2.45, y = 7.91$
Search space	Limited/countable	Very large or infinite
Actions	Move tile left/right	Increase/decrease value
Algorithms	BFS, DFS, A*	Gradient descent, Newton method

8. Methods Used in Continuous Local Search

8.1 Gradient Descent

Gradient Descent is one of the most common methods used in continuous search.

It is used to minimize a function.

Formula:

$$x_{\text{new}} = x_{\text{old}} - \alpha \times \text{gradient}$$

Where:

Symbol Meaning

x_{new} Updated value

x_{old} Current value

α Learning rate / step size

gradient Direction of steepest increase

Simple meaning:

Gradient descent moves in the direction where the error decreases.

8.2 Newton's Method

Newton's method is another optimization technique.

It uses:

- First derivative

- Second derivative

It can reach the optimum faster, but it requires more calculation.

8.3 Random Restart

Sometimes local search gets stuck at a local optimum.

So the algorithm starts again from another random point.

This is called **random restart**.

Example:

Start from $x = 10$

If stuck, restart from $x = -5$

If stuck again, restart from $x = 3$

8.4 Simulated Annealing

Simulated annealing sometimes accepts a worse solution temporarily.

This helps the algorithm escape from local optimum.

9. Local Optimum Problem

A **local optimum** is a solution that is better than nearby solutions but not the best overall.

Example:

Small hill = local maximum

Highest hill = global maximum

Local search may stop at the small hill and miss the highest hill.

To reduce this problem, we use:

Random restart

Simulated annealing

Better step size

10. Real-Life Example in Machine Learning

In machine learning, a model has parameters such as:

UNIT-I

weight = 0.45

bias = 1.20

The goal is to reduce prediction error.

The algorithm changes the values slowly:

weight = 0.45 $\rightarrow$ 0.42 $\rightarrow$ 0.39 $\rightarrow$ 0.35

If the error decreases, the new value is accepted.

This is an example of **local search in continuous spaces**.

11. Applications

Area	Use
Machine Learning	Optimizing weights and bias
Deep Learning	Training neural networks
Robotics	Motion planning and control
Self-driving Cars	Speed and steering control
Engineering	Design optimization
Economics	Cost and profit optimization

12. Advantages

Advantage	Explanation
Works with real values	Useful for decimal values
Efficient	Does not search all possibilities
Useful in ML/DL	Helps train models
Simple idea	Improves solution step by step

13. Limitations

UNIT-I

Limitation	Explanation
Can get stuck	May stop at local optimum
Depends on starting point	Different starts may give different results
Step size problem	Too large or too small step affects result
No guarantee always	May not always find global optimum